



Atelier Python — Suivi de Budget Personnel



Par Robert DIASSÉ

Application graphique avec Tkinter & Matplotlib

Niveau : Débutant (vous connaissez les variables, listes, dictionnaires, fonctions et boucles)

Durée estimée : 3h à 4h

Résultat final : Une application de bureau complète avec interface graphique, graphiques dynamiques et sauvegarde automatique

Objectifs pédagogiques

À la fin de cet atelier, vous serez capable de :

- Créer une interface graphique professionnelle avec **Tkinter**
 - Afficher des **graphiques dynamiques** intégrés à une application (matplotlib)
 - Lire et écrire des données dans un **fichier JSON**
 - Organiser un projet Python en **fonctions et modules**
 - Comprendre le concept de **callback** (fonction déclenchée par un événement)
-

Ce que vous allez construire

Une application de suivi de budget avec :

- ☒ Formulaire pour ajouter des revenus et dépenses
 - ☒ Liste colorée des transactions (vert = revenu, rouge = dépense)
 - ☒ Affichage du solde en temps réel avec alerte visuelle
 - ☒ Graphique camembert des dépenses par catégorie
 - ☒ Graphique en barres revenus vs dépenses
 - ☒ Sauvegarde automatique dans un fichier JSON
 - ☒ Bouton de suppression de transaction
-

Étape 0 — Préparation de l'environnement

0.1 Vérifier Python

Ouvrez un terminal et tapez :

```
python --version
```

Vous devez voir **Python 3.8** ou supérieur. Si Python n'est pas installé, téléchargez-le sur python.org.

0.2 Installer les bibliothèques nécessaires

Tkinter est **déjà inclus** avec Python. Il faut seulement installer **matplotlib** pour les graphiques.

```
pip install matplotlib
```

Pourquoi matplotlib ? C'est la bibliothèque de référence pour créer des graphiques en Python. Elle peut s'intégrer directement dans une fenêtre Tkinter.

0.3 Créer la structure du projet

Créez un dossier **budget_app/** et à l'intérieur, créez un seul fichier :

```
budget_app/  
└─ app.py
```

Ouvrez **app.py** dans votre éditeur (VS Code, Thonny, PyCharm...).

✓ Vérification étape 0

Dans **app.py**, écrivez ces deux lignes et exécutez le fichier :

```
import tkinter  
import matplotlib  
print("Tout est prêt !")
```

Si vous voyez **Tout est prêt !** sans erreur, passez à l'étape suivante.

En cas d'erreur sur **matplotlib**, relancez **pip install matplotlib**.

Étape 1 — La fenêtre principale

1.1 Créer une fenêtre Tkinter de base

Effacez le contenu de **app.py** et écrivez le code suivant :

```
import tkinter as tk
from tkinter import ttk

# — Création de la fenêtre principale —————
fenetre = tk.Tk()

# Titre affiché dans la barre de la fenêtre
fenetre.title(" MonBudget – Suivi Personnel")

# Taille de départ : 900 pixels de large, 650 pixels de haut
fenetre.geometry("900x650")

# Empêche de réduire la fenêtre en dessous de cette taille minimale
fenetre.minsize(800, 600)

# Couleur de fond de la fenêtre
fenetre.configure(bg="#1e1e2e")

# — Lancement de la boucle principale —————
# Cette ligne DOIT toujours être la dernière : elle maintient la fenêtre ouverte
fenetre.mainloop()
```

Exécutez avec `python app.py`. Une fenêtre sombre doit apparaître.

1.2 Comprendre la structure Tkinter

Notion clé — La boucle principale

`fenetre.mainloop()` est une boucle infinie qui attend vos actions (clic, saisie...).

Chaque fois que vous interagissez, Tkinter réagit en appelant des **fonctions de rappel (callbacks)**.

C'est le principe de toute interface graphique.

1.3 Diviser la fenêtre en deux panneaux

On va créer une mise en page en deux colonnes :

- **Gauche** : formulaire de saisie + liste des transactions
- **Droite** : graphiques

```
import tkinter as tk
from tkinter import ttk

fenetre = tk.Tk()
fenetre.title(" MonBudget – Suivi Personnel")
fenetre.geometry("900x650")
fenetre.minsize(800, 600)
fenetre.configure(bg="#1e1e2e")
```

```
# — Panneau gauche —
# width=320 : largeur fixe en pixels
# bg : couleur de fond
# padx/pady : marge intérieure horizontale/verticale
panneau_gauche = tk.Frame(fenetre, width=320, bg="#2a2a3e", padx=10, pady=10)
# fill=tk.Y : occupe toute la hauteur disponible
# side=tk.LEFT : se colle à gauche
panneau_gauche.pack(fill=tk.Y, side=tk.LEFT)

# — Panneau droit —
panneau_droit = tk.Frame(fenetre, bg="#1e1e2e", padx=10, pady=10)
# expand=True : prend tout l'espace restant
# fill=tk.BOTH : dans les deux directions
panneau_droit.pack(fill=tk.BOTH, expand=True, side=tk.LEFT)

# Titres temporaires pour visualiser la mise en page
tk.Label(panneau_gauche, text="PANNEAU GAUCHE", bg="#2a2a3e",
        fg="white").pack()
tk.Label(panneau_droit, text="PANNEAU DROIT", bg="#1e1e2e",
        fg="white").pack()

fenetre.mainloop()
```

✓ Vérification étape 1

Vous devez voir une fenêtre avec deux zones distinctes : un panneau gauche légèrement plus clair et un panneau droit plus sombre. Si c'est bon, continuez.

Étape 2 — La structure de données

Avant de construire l'interface, définissons les données que l'application va manipuler.

2.1 La liste des transactions

Une transaction est un dictionnaire Python. Exemple :

```
{
    "description": "Salaire",
    "montant": 1500.0,
    "type": "revenu",          # "revenu" ou "depense"
    "categorie": "Salaire",
    "date": "2024-03-15"
}
```

Une **liste** de ces dictionnaires contiendra toutes nos transactions.

2.2 Les catégories disponibles

```
CATEGORIES = [
    "Salaire", "Freelance", "Autre revenu",    # Revenus
    "Alimentation", "Transport", "Logement",  # Dépenses essentielles
    "Santé", "Loisirs", "Vêtements",         # Dépenses secondaires
    "Épargne", "Autre dépense"
]
```

2.3 Ajouter les données au projet

Remplacez le contenu de `app.py` par ce code (les imports et les données) :

```
import tkinter as tk
from tkinter import ttk, messagebox
import json
import os
from datetime import date
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

# =====
# DONNÉES DE L'APPLICATION
# =====

# Catégories disponibles dans le formulaire
CATEGORIES = [
    "Salaire", "Freelance", "Autre revenu",
    "Alimentation", "Transport", "Logement",
    "Santé", "Loisirs", "Vêtements",
    "Épargne", "Autre dépense"
]

# Fichier où les transactions seront sauvegardées
FICHIER_DONNEES = "transactions.json"

# Budget d'alerte : un avertissement s'affichera si les dépenses dépassent ce
# seuil
BUDGET_ALERTE = 1000.0

# Liste qui contiendra toutes les transactions pendant l'exécution du programme
transactions = []

# =====
# FENÊTRE PRINCIPALE
# =====

fenetre = tk.Tk()
fenetre.title(" MonBudget – Suivi Personnel")
fenetre.geometry("950x700")
fenetre.minsize(800, 600)
fenetre.configure(bg="#1e1e2e")
```

```
print("Application démarrée.")
fenetre.mainloop()
```

✓ Vérification étape 2

La fenêtre s'ouvre et le terminal affiche **Application démarrée**. Aucune erreur. ✓

Étape 3 — Le formulaire de saisie

3.1 Construire le formulaire dans le panneau gauche

On va ajouter : un titre, des champs de saisie et un bouton.

Ajoutez ce code **avant** `fenetre.mainloop()` :

```
# =====
# MISE EN PAGE – DEUX PANNEAUX
# =====

panneau_gauche = tk.Frame(fenetre, width=330, bg="#2a2a3e", padx=12, pady=12)
panneau_gauche.pack(fill=tk.Y, side=tk.LEFT)
# pack_propagate(False) : empêche le panneau de rétrécir selon son contenu
panneau_gauche.pack_propagate(False)

panneau_droit = tk.Frame(fenetre, bg="#1e1e2e", padx=10, pady=10)
panneau_droit.pack(fill=tk.BOTH, expand=True, side=tk.LEFT)

# =====
# ZONE DE SOLDE (en haut à gauche)
# =====

# Cadre pour le solde
cadre_solde = tk.Frame(panneau_gauche, bg="#3a3a5e", pady=10, padx=10,
                      relief=tk.RIDGE, bd=1)
cadre_solde.pack(fill=tk.X, pady=(0, 12))

tk.Label(cadre_solde, text="SOLDE ACTUEL", bg="#3a3a5e",
        fg="#a0a0c0", font=("Helvetica", 9)).pack()

# Label du solde – on gardera une référence pour le mettre à jour dynamiquement
label_solde = tk.Label(cadre_solde, text="0.00 FCFA", bg="#3a3a5e",
                      fg="#50fa7b", font=("Helvetica", 20, "bold"))
label_solde.pack()

# =====
# FORMULAIRE D'AJOUT DE TRANSACTION
# =====

cadre_form = tk.LabelFrame(panneau_gauche, text=" Nouvelle transaction ",
                          bg="#2a2a3e", fg="#bd93f9",
```

```

        font=("Helvetica", 10, "bold"),
        padx=8, pady=8)
cadre_form.pack(fill=tk.X, pady=(0, 10))

# — Description —
tk.Label(cadre_form, text="Description :", bg="#2a2a3e", fg="#cdd6f4",
        font=("Helvetica", 9)).pack(anchor=tk.W)

# StringVar : variable Tkinter liée à un champ texte
var_description = tk.StringVar()
champ_description = tk.Entry(cadre_form, textvariable=var_description,
                             bg="#44475a", fg="white", insertbackground="white",
                             relief=tk.FLAT, font=("Helvetica", 10))
champ_description.pack(fill=tk.X, pady=(2, 8))

# — Montant —
tk.Label(cadre_form, text="Montant (FCFA) :", bg="#2a2a3e", fg="#cdd6f4",
        font=("Helvetica", 9)).pack(anchor=tk.W)

var_montant = tk.StringVar()
champ_montant = tk.Entry(cadre_form, textvariable=var_montant,
                         bg="#44475a", fg="white", insertbackground="white",
                         relief=tk.FLAT, font=("Helvetica", 10))
champ_montant.pack(fill=tk.X, pady=(2, 8))

# — Catégorie —
tk.Label(cadre_form, text="Catégorie :", bg="#2a2a3e", fg="#cdd6f4",
        font=("Helvetica", 9)).pack(anchor=tk.W)

var_categorie = tk.StringVar(value=CATEGORIES[0])
# OptionMenu : liste déroulante – *CATEGORIES déplie la liste comme arguments
menu_categorie = tk.OptionMenu(cadre_form, var_categorie, *CATEGORIES)
menu_categorie.configure(bg="#44475a", fg="white", relief=tk.FLAT,
                        font=("Helvetica", 10), activebackground="#6272a4",
                        highlightthickness=0)
menu_categorie["menu"].configure(bg="#44475a", fg="white")
menu_categorie.pack(fill=tk.X, pady=(2, 8))

# — Type de transaction —
tk.Label(cadre_form, text="Type :", bg="#2a2a3e", fg="#cdd6f4",
        font=("Helvetica", 9)).pack(anchor=tk.W)

# StringVar : variable pour les boutons radio
var_type = tk.StringVar(value="revenu")

cadre_radio = tk.Frame(cadre_form, bg="#2a2a3e")
cadre_radio.pack(anchor=tk.W, pady=(2, 8))

tk.Radiobutton(cadre_radio, text=" Revenu", variable=var_type,
               value="revenu", bg="#2a2a3e", fg="#50fa7b",
               selectcolor="#44475a", activebackground="#2a2a3e",
               font=("Helvetica", 10)).pack(side=tk.LEFT, padx=(0, 10))

tk.Radiobutton(cadre_radio, text=" Dépense", variable=var_type,

```

```

        value="depense", bg="#2a2a3e", fg="#ff5555",
        selectcolor="#44475a", activebackground="#2a2a3e",
        font=("Helvetica", 10)).pack(side=tk.LEFT)

# — Bouton Ajouter —
# command= : la fonction à appeler quand on clique (définie à l'étape 4)
btn_ajouter = tk.Button(cadre_form, text="  Ajouter la transaction",
                        bg="#6272a4", fg="white",
                        font=("Helvetica", 10, "bold"),
                        relief=tk.FLAT, cursor="hand2",
                        activebackground="#7282b4")
btn_ajouter.pack(fill=tk.X, pady=(4, 0))

```

✓ Vérification étape 3

Relancez l'application. Vous devez voir le formulaire complet dans le panneau gauche : champ description, montant, catégorie, boutons radio et bouton Ajouter. Le bouton ne fait encore rien. ✓

Étape 4 — La liste des transactions

4.1 Créer la liste (Treeview)

Le widget `Treeview` de Tkinter permet d'afficher des données en tableau avec des colonnes. Ajoutez ce code après le formulaire, toujours **avant** `mainloop()` :

```

# =====
# LISTE DES TRANSACTIONS (Treeview = tableau)
# =====

cadre_liste = tk.LabelFrame(panneau_gauche, text="  Transactions  ",
                            bg="#2a2a3e", fg="#bd93f9",
                            font=("Helvetica", 10, "bold"),
                            padx=8, pady=8)
cadre_liste.pack(fill=tk.BOTH, expand=True)

# Style personnalisé pour le Treeview (tableau)
style = ttk.Style()
style.theme_use("clam") # Thème de base nécessaire pour la personnalisation
style.configure("Custom.Treeview",
                background="#2a2a3e",
                foreground="#cdd6f4",
                fieldbackground="#2a2a3e",
                rowheight=26,
                font=("Helvetica", 9))
style.configure("Custom.Treeview.Heading",
                background="#44475a",
                foreground="#bd93f9",
                font=("Helvetica", 9, "bold"))

# Couleur au survol d'une ligne

```



```

style.map("Custom.Treeview",
          background=[("selected", "#6272a4")])

# Treeview avec 3 colonnes (sans la colonne #0 qui est l'icône d'arbre)
tableau = ttk.Treeview(cadre_liste,
                       columns=("description", "montant", "date"),
                       show="headings",      # Masque la colonne d'icône
                       style="Custom.Treeview",
                       height=8)

# Définition des en-têtes de colonnes
tableau.heading("description", text="Description")
tableau.heading("montant", text="Montant")
tableau.heading("date", text="Date")

# Largeur de chaque colonne
tableau.column("description", width=130, anchor=tk.W)
tableau.column("montant", width=90, anchor=tk.CENTER)
tableau.column("date", width=80, anchor=tk.CENTER)

# Scrollbar verticale liée au tableau
scrollbar = ttk.Scrollbar(cadre_liste, orient=tk.VERTICAL,
                          command=tableau.yview)
tableau.configure(yscrollcommand=scrollbar.set)

tableau.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

# — Bouton Supprimer —
btn_supprimer = tk.Button(panneau_gauche,
                           text="  Supprimer la sélection",
                           bg="#ff5555", fg="white",
                           font=("Helvetica", 9, "bold"),
                           relief=tk.FLAT, cursor="hand2",
                           activebackground="#ff6e6e")
btn_supprimer.pack(fill=tk.X, pady=(6, 0))

```

4.2 Définir les fonctions de logique

Maintenant, on définit les fonctions qui gèrent l'ajout, la suppression et la mise à jour.

Ajoutez ce bloc **avant la création des panneaux** (après les données) :

```

# =====
# FONCTIONS DE L'APPLICATION
# =====

def calculer_solde():
    """Calcule et retourne (total_revenus, total_depenses, solde)."""
    revenus = sum(t["montant"] for t in transactions if t["type"] == "revenu")
    depenses = sum(t["montant"] for t in transactions if t["type"] == "depense")

```

```

    return revenus, depenses, revenus - depenses

def mettre_a_jour_solde():
    """Met à jour l'affichage du solde en couleur."""
    revenus, depenses, solde = calculer_solde()
    label_solde.config(text=f"{solde:,.0f} FCFA")

    # Couleur dynamique selon le solde
    if solde > 0:
        label_solde.config(fg="#50fa7b")    # Vert : solde positif
    elif solde == 0:
        label_solde.config(fg="#f1fa8c")    # Jaune : solde nul
    else:
        label_solde.config(fg="#ff5555")    # Rouge : solde négatif

    # Alerte si les dépenses dépassent le budget d'alerte
    if depenses > BUDGET_ALERTE:
        messagebox.showwarning(
            " Alerte Budget",
            f"Vos dépenses totales ({depenses:,.0f} FCFA) "
            f"dépassent votre seuil d'alerte ({BUDGET_ALERTE:,.0f} FCFA) !"
        )

def rafraichir_tableau():
    """Vide et reconstruit le tableau des transactions."""
    # Supprimer toutes les lignes existantes
    for ligne in tableau.get_children():
        tableau.delete(ligne)

    # Reparcourir la liste et recréer chaque ligne
    for i, t in enumerate(transactions):
        montant_str = f"+{t['montant']:,.0f}" if t["type"] == "revenu" \
            else f"-{t['montant']:,.0f}"

        # Insérer la ligne dans le tableau
        # iid : identifiant unique de la ligne (on utilise l'indice)
        item = tableau.insert("", tk.END, iid=str(i),
                               values=(t["description"], montant_str, t["date"]))

        # Colorer la ligne selon le type
        if t["type"] == "revenu":
            tableau.item(item, tags=("revenu",))
        else:
            tableau.item(item, tags=("depense",))

    # Appliquer les couleurs par tag
    tableau.tag_configure("revenu", foreground="#50fa7b")
    tableau.tag_configure("depense", foreground="#ff5555")

def ajouter_transaction():
    """Lit le formulaire, valide les données et ajoute la transaction."""

```

```

description = var_description.get().strip()
montant_str = var_montant.get().strip()
categorie = var_categorie.get()
type_t = var_type.get()

# — Validation des champs —————
if not description:
    messagebox.showerror("Erreur", "Veuillez saisir une description.")
    return

# Vérification que le montant est bien un nombre positif
try:
    montant = float(montant_str)
    if montant <= 0:
        raise ValueError
except ValueError:
    messagebox.showerror("Erreur", "Le montant doit être un nombre positif.")
    return

# — Créer la transaction et l'ajouter à la liste —————
nouvelle = {
    "description": description,
    "montant": montant,
    "type": type_t,
    "categorie": categorie,
    "date": str(date.today()) # Date du jour au format YYYY-MM-DD
}
transactions.append(nouvelle)

# — Mettre à jour l'interface —————
rafraichir_tableau()
mettre_a_jour_solde()
rafraichir_graphiques() # Définie à l'étape suivante
sauvegarder_donnees()   # Définie à l'étape suivante

# Vider les champs du formulaire
var_description.set("")
var_montant.set("")

# Remettre le focus sur le champ description pour saisir rapidement
champ_description.focus()

def supprimer_transaction():
    """Supprime la transaction sélectionnée dans le tableau."""
    selection = tableau.selection() # Retourne les iid sélectionnés
    if not selection:
        messagebox.showinfo("Info", "Sélectionnez une transaction à supprimer.")
        return

    # Confirmation avant suppression
    reponse = messagebox.askyesno(
        "Confirmer", "Supprimer cette transaction ?"
    )

```

```

if not reponse:
    return

# Supprimer dans la liste (l'iid est l'indice de la transaction)
indice = int(selection[0])
transactions.pop(indice)

# Mettre à jour l'interface
rafraichir_tableau()
mettre_a_jour_solde()
rafraichir_graphiques()
sauvegarder_donnees()

```

4.3 Connecter les boutons aux fonctions

Après avoir créé les boutons dans l'interface, modifiez les lignes `btn_ajouter` et `btn_supprimer` pour leur ajouter le paramètre `command=` :

```

# Dans le formulaire (étape 3), remplacez la ligne btn_ajouter par :
btn_ajouter = tk.Button(cadre_form, text="  Ajouter la transaction",
                        bg="#6272a4", fg="white",
                        font=("Helvetica", 10, "bold"),
                        relief=tk.FLAT, cursor="hand2",
                        activebackground="#7282b4",
                        command=lambda: ajouter_transaction())  # ← AJOUT

# Et la ligne btn_supprimer par :
btn_supprimer = tk.Button(panneau_gauche,
                          text="  Supprimer la sélection",
                          bg="#ff5555", fg="white",
                          font=("Helvetica", 9, "bold"),
                          relief=tk.FLAT, cursor="hand2",
                          activebackground="#ff6e6e",
                          command=lambda: supprimer_transaction())  # ← AJOUT

```

Notion clé — Les callbacks

`command=ajouter_transaction` signifie : *"quand on clique, appelle cette fonction"*.

On passe le **nom** de la fonction (sans parenthèses), pas son résultat.

C'est la base du fonctionnement des interfaces graphiques.

✓ Vérification étape 4

Relancez l'application. Saisissez une description, un montant, choisissez une catégorie et cliquez sur Ajouter. La ligne doit apparaître dans le tableau (en vert ou rouge) et le solde doit se mettre à jour. ✓

Étape 5 — Les graphiques avec Matplotlib

5.1 Comprendre l'intégration Matplotlib/Tkinter

Notion clé — FigureCanvasTkAgg

Matplotlib génère des graphiques dans une "figure". Normalement, il affiche cette figure dans sa propre fenêtre.

`FigureCanvasTkAgg` est un adaptateur qui transforme cette figure en **widget Tkinter** qu'on peut placer dans notre interface.

5.2 Créer la zone des graphiques

Ajoutez ce code après la création de `panneau_droit` :

```
# =====
# ZONE GRAPHIQUES (panneau droit)
# =====

# Titre du panneau droit
tk.Label(panneau_droit, text=" Analyse du budget",
        bg="#1e1e2e", fg="#bd93f9",
        font=(Helvetica, 13, "bold")).pack(pady=(0, 8))

# — Création de la figure Matplotlib —
# figsize : taille en pouces (largeur, hauteur)
# facecolor : couleur de fond de la figure
figure = plt.Figure(figsize=(5.5, 5.5), facecolor="#1e1e2e")

# Deux sous-graphiques (subplots) : 2 lignes, 1 colonne
# ax1 : graphique du haut (camembert)
# ax2 : graphique du bas (barres)
ax1 = figure.add_subplot(211) # 2 lignes, 1 col, position 1
ax2 = figure.add_subplot(212) # 2 lignes, 1 col, position 2

# — Adapter le fond de chaque sous-graphique —
ax1.set_facecolor("#2a2a3e")
ax2.set_facecolor("#2a2a3e")

# — Canvas : le widget Tkinter qui affiche la figure —
canvas = FigureCanvasTkAgg(figure, master=panneau_droit)
canvas.draw()
canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
```

5.3 Définir la fonction de mise à jour des graphiques

Ajoutez cette fonction dans le bloc des fonctions (avec les autres) :

```
def rafraichir_graphiques():
    """Redessine les deux graphiques selon les transactions actuelles."""

    # Effacer les graphiques précédents
    ax1.clear()
```

```

ax2.clear()

# Réappliquer le fond sombre après clear()
ax1.set_facecolor("#2a2a3e")
ax2.set_facecolor("#2a2a3e")

# — Graphique 1 : Camembert des dépenses par catégorie —————
depenses_par_cat = {}
for t in transactions:
    if t["type"] == "depense":
        cat = t["categorie"]
        # dict.get(clé, valeur_par_défaut) : retourne 0 si la clé n'existe pas
        depenses_par_cat[cat] = depenses_par_cat.get(cat, 0) + t["montant"]

if depenses_par_cat:
    # Couleurs du graphique (palette cohérente avec le thème sombre)
    couleurs = ["#ff5555", "#ff79c6", "#bd93f9", "#6272a4",
                "#ffb86c", "#50fa7b", "#8be9fd", "#f1fa8c",
                "#ff6e6e", "#caa9fa", "#69ff47"]

    wedges, texts, autotexts = ax1.pie(
        depenses_par_cat.values(),
        labels=depenses_par_cat.keys(),
        autopct="%1.0f%%",          # Afficher le pourcentage
        colors=couleurs[:len(depenses_par_cat)],
        textprops={"color": "#cdd6f4", "fontsize": 7},
        startangle=90
    )
    # Couleur des pourcentages
    for at in autotexts:
        at.set_color("#f8f8f2")
        at.set_fontsize(7)

    ax1.set_title("Dépenses par catégorie",
                  color="#cdd6f4", fontsize=9, pad=8)
else:
    # Message si pas de données
    ax1.text(0.5, 0.5, "Aucune dépense",
            ha="center", va="center",
            color="#6272a4", fontsize=10,
            transform=ax1.transAxes)
    ax1.set_title("Dépenses par catégorie",
                  color="#cdd6f4", fontsize=9)

# — Graphique 2 : Barres Revenus vs Dépenses —————
revenus, depenses, _ = calculer_solde()

barres = ax2.bar(
    ["Revenus", "Dépenses"],
    [revenus, depenses],
    color=["#50fa7b", "#ff5555"],
    width=0.5,
    edgecolor="none"
)

```

```

# Afficher la valeur au-dessus de chaque barre
for barre in barres:
    hauteur = barre.get_height()
    ax2.text(
        barre.get_x() + barre.get_width() / 2,  # Position X : centre de la
barre
        hauteur + (max(revenus, depenses) * 0.02), # Position Y : au-dessus
        f"{hauteur:,.0f}",
        ha="center", va="bottom",
        color="#cdd6f4", fontsize=8
    )

ax2.set_title("Revenus vs Dépenses (FCFA)",
              color="#cdd6f4", fontsize=9)
ax2.tick_params(colors="#cdd6f4", labels=8)

# Couleur des bordures du graphique
for spine in ax2.spines.values():
    spine.set_edgecolor("#44475a")

# Ajustement automatique des marges pour éviter les chevauchements
figure.tight_layout(pad=1.5)

# Demander au canvas de se redessiner avec les nouvelles données
canvas.draw()

```

✓ Vérification étape 5

Relancez et ajoutez plusieurs transactions (revenus et dépenses de catégories différentes). Les graphiques doivent se mettre à jour à chaque ajout. Le camembert doit montrer les proportions de dépenses, les barres montrent revenus vs dépenses. ✓

Étape 6 — La sauvegarde en JSON

Sans sauvegarde, toutes les données sont perdues à la fermeture. On va utiliser le format **JSON** pour persister les transactions dans un fichier.

Notion clé — JSON

JSON (JavaScript Object Notation) est un format texte universel pour stocker des données structurées. Python convertit automatiquement ses listes et dictionnaires en JSON avec le module `json` (déjà importé).

6.1 Fonctions de sauvegarde et chargement

Ajoutez ces deux fonctions dans le bloc des fonctions :

```
def sauvegarder_donnees():
    """Écrit la liste des transactions dans le fichier JSON."""
    try:
        # open() ouvre le fichier en mode écriture ("w")
        # encoding="utf-8" : nécessaire pour les accents
        with open(FICHIER_DONNEES, "w", encoding="utf-8") as f:
            # indent=2 : indentation pour que le fichier soit lisible
            json.dump(transactions, f, ensure_ascii=False, indent=2)
    except Exception as e:
        messagebox.showerror("Erreur", f"Impossible de sauvegarder :\n{e}")

def charger_donnees():
    """Charge les transactions depuis le fichier JSON (si il existe)."""
    # os.path.exists() : vérifie si le fichier existe déjà
    if not os.path.exists(FICHIER_DONNEES):
        return # Première utilisation, rien à charger

    try:
        with open(FICHIER_DONNEES, "r", encoding="utf-8") as f:
            donnees = json.load(f) # Convertit le JSON en liste Python

        # Ajouter chaque transaction dans notre liste globale
        for t in donnees:
            transactions.append(t)

    except Exception as e:
        messagebox.showerror("Erreur", f"Impossible de charger les données :\n{e}")
```

6.2 Appeler le chargement au démarrage

Ajoutez ces lignes **juste avant** `fenetre.mainloop()` :

```
# =====
# INITIALISATION – Chargement des données et premier affichage
# =====

charger_donnees()      # Charger les transactions sauvegardées
rafraichir_tableau()   # Afficher dans le tableau
mettre_a_jour_solde()  # Mettre à jour le solde
rafraichir_graphiques() # Dessiner les graphiques
```

✓ Vérification étape 6

Ajoutez des transactions, fermez l'application, puis rouvrez-la. Les transactions doivent réapparaître. Vérifiez aussi qu'un fichier `transactions.json` a été créé dans votre dossier. Ouvrez-le dans un éditeur texte pour voir sa structure. ✓

Étape 7 — Code final complet et nettoyé

Voici le **code complet et organisé** de l'application. Copiez-le intégralement dans votre `app.py` :

```
#
# MonBudget – Suivi de Budget Personnel
# Atelier Python ISI – Application Tkinter + Matplotlib
#

import tkinter as tk
from tkinter import ttk, messagebox
import json
import os
from datetime import date
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

# =====
# CONFIGURATION
# =====

CATEGORIES = [
    "Salaire", "Freelance", "Autre revenu",
    "Alimentation", "Transport", "Logement",
    "Santé", "Loisirs", "Vêtements",
    "Épargne", "Autre dépense"
]
FICHIER_DONNEES = "transactions.json"
BUDGET_ALERTE = 1000.0
transactions = []

# =====
# FENÊTRE PRINCIPALE
# =====

fenetre = tk.Tk()
fenetre.title(" MonBudget – Suivi Personnel")
fenetre.geometry("950x700")
fenetre.minsize(800, 600)
fenetre.configure(bg="#1e1e2e")

# =====
# MISE EN PAGE – DEUX PANNEAUX
# =====

panneau_gauche = tk.Frame(fenetre, width=330, bg="#2a2a3e", padx=12, pady=12)
panneau_gauche.pack(fill=tk.Y, side=tk.LEFT)
panneau_gauche.pack_propagate(False)

panneau_droit = tk.Frame(fenetre, bg="#1e1e2e", padx=10, pady=10)
panneau_droit.pack(fill=tk.BOTH, expand=True, side=tk.LEFT)
```

```

# =====
# WIDGETS – PANNEAU GAUCHE
# =====

# — Solde —————
cadre_solde = tk.Frame(panneau_gauche, bg="#3a3a5e", pady=10, padx=10,
                      relief=tk.RIDGE, bd=1)
cadre_solde.pack(fill=tk.X, pady=(0, 12))
tk.Label(cadre_solde, text="SOLDE ACTUEL", bg="#3a3a5e",
        fg="#a0a0c0", font=("Helvetica", 9)).pack()
label_solde = tk.Label(cadre_solde, text="0 FCFA", bg="#3a3a5e",
                      fg="#50fa7b", font=("Helvetica", 20, "bold"))
label_solde.pack()

# — Formulaire —————
cadre_form = tk.LabelFrame(panneau_gauche, text=" Nouvelle transaction ",
                          bg="#2a2a3e", fg="#bd93f9",
                          font=("Helvetica", 10, "bold"), padx=8, pady=8)
cadre_form.pack(fill=tk.X, pady=(0, 10))

tk.Label(cadre_form, text="Description :", bg="#2a2a3e",
        fg="#cdd6f4", font=("Helvetica", 9)).pack(anchor=tk.W)
var_description = tk.StringVar()
champ_description = tk.Entry(cadre_form, textvariable=var_description,
                            bg="#44475a", fg="white",
                            insertbackground="white",
                            relief=tk.FLAT, font=("Helvetica", 10))
champ_description.pack(fill=tk.X, pady=(2, 8))

tk.Label(cadre_form, text="Montant (FCFA) :", bg="#2a2a3e",
        fg="#cdd6f4", font=("Helvetica", 9)).pack(anchor=tk.W)
var_montant = tk.StringVar()
champ_montant = tk.Entry(cadre_form, textvariable=var_montant,
                        bg="#44475a", fg="white",
                        insertbackground="white",
                        relief=tk.FLAT, font=("Helvetica", 10))
champ_montant.pack(fill=tk.X, pady=(2, 8))

tk.Label(cadre_form, text="Catégorie :", bg="#2a2a3e",
        fg="#cdd6f4", font=("Helvetica", 9)).pack(anchor=tk.W)
var_categorie = tk.StringVar(value=CATEGORIES[0])
menu_categorie = tk.OptionMenu(cadre_form, var_categorie, *CATEGORIES)
menu_categorie.configure(bg="#44475a", fg="white", relief=tk.FLAT,
                        font=("Helvetica", 10), activebackground="#6272a4",
                        highlightthickness=0)
menu_categorie["menu"].configure(bg="#44475a", fg="white")
menu_categorie.pack(fill=tk.X, pady=(2, 8))

tk.Label(cadre_form, text="Type :", bg="#2a2a3e",
        fg="#cdd6f4", font=("Helvetica", 9)).pack(anchor=tk.W)
var_type = tk.StringVar(value="revenu")
cadre_radio = tk.Frame(cadre_form, bg="#2a2a3e")
cadre_radio.pack(anchor=tk.W, pady=(2, 8))

```

```

tk.Radiobutton(cadre_radio, text=" Revenu", variable=var_type,
               value="revenu", bg="#2a2a3e", fg="#50fa7b",
               selectcolor="#44475a", activebackground="#2a2a3e",
               font=("Helvetica", 10)).pack(side=tk.LEFT, padx=(0, 10))
tk.Radiobutton(cadre_radio, text=" Dépense", variable=var_type,
               value="depense", bg="#2a2a3e", fg="#ff5555",
               selectcolor="#44475a", activebackground="#2a2a3e",
               font=("Helvetica", 10)).pack(side=tk.LEFT)

btn_ajouter = tk.Button(cadre_form, text=" Ajouter la transaction",
                        bg="#6272a4", fg="white",
                        font=("Helvetica", 10, "bold"), relief=tk.FLAT,
                        cursor="hand2", activebackground="#7282b4",
                        command=lambda: ajouter_transaction())
btn_ajouter.pack(fill=tk.X, pady=(4, 0))

# — Tableau des transactions —————
cadre_liste = tk.LabelFrame(panneau_gauche, text=" Transactions ",
                             bg="#2a2a3e", fg="#bd93f9",
                             font=("Helvetica", 10, "bold"), padx=8, pady=8)
cadre_liste.pack(fill=tk.BOTH, expand=True)

style = ttk.Style()
style.theme_use("clam")
style.configure("Custom.Treeview",
                background="#2a2a3e", foreground="#cdd6f4",
                fieldbackground="#2a2a3e", rowheight=26,
                font=("Helvetica", 9))
style.configure("Custom.Treeview.Heading",
                background="#44475a", foreground="#bd93f9",
                font=("Helvetica", 9, "bold"))
style.map("Custom.Treeview",
          background=[("selected", "#6272a4")])

tableau = ttk.Treeview(cadre_liste,
                       columns=("description", "montant", "date"),
                       show="headings", style="Custom.Treeview", height=8)
tableau.heading("description", text="Description")
tableau.heading("montant", text="Montant")
tableau.heading("date", text="Date")
tableau.column("description", width=130, anchor=tk.W)
tableau.column("montant", width=90, anchor=tk.CENTER)
tableau.column("date", width=80, anchor=tk.CENTER)

scrollbar = ttk.Scrollbar(cadre_liste, orient=tk.VERTICAL,
                          command=tableau.yview)
tableau.configure(yscrollcommand=scrollbar.set)
tableau.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

btn_supprimer = tk.Button(panneau_gauche,
                           text=" Supprimer la sélection",
                           bg="#ff5555", fg="white",
                           font=("Helvetica", 9, "bold"), relief=tk.FLAT,

```

```

        cursor="hand2", activebackground="#ff6e6e",
        command=lambda: supprimer_transaction())
btn_supprimer.pack(fill=tk.X, pady=(6, 0))

# =====
# WIDGETS – PANNEAU DROIT (Graphiques)
# =====

tk.Label(panneau_droit, text=" Analyse du budget",
        bg="#1e1e2e", fg="#bd93f9",
        font=("Helvetica", 13, "bold")).pack(pady=(0, 8))

figure = plt.Figure(figsize=(5.5, 5.5), facecolor="#1e1e2e")
ax1 = figure.add_subplot(211)
ax2 = figure.add_subplot(212)
ax1.set_facecolor("#2a2a3e")
ax2.set_facecolor("#2a2a3e")

canvas = FigureCanvasTkAgg(figure, master=panneau_droit)
canvas.draw()
canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

# =====
# FONCTIONS DE LOGIQUE
# =====

def calculer_solde():
    revenus = sum(t["montant"] for t in transactions if t["type"] == "revenu")
    depenses = sum(t["montant"] for t in transactions if t["type"] == "depense")
    return revenus, depenses, revenus - depenses

def mettre_a_jour_solde():
    revenus, depenses, solde = calculer_solde()
    label_solde.config(text=f"{solde:,.0f} FCFA")
    if solde > 0:
        label_solde.config(fg="#50fa7b")
    elif solde == 0:
        label_solde.config(fg="#f1fa8c")
    else:
        label_solde.config(fg="#ff5555")
    if depenses > BUDGETALERTE:
        messagebox.showwarning(
            " Alerte Budget",
            f"Vos dépenses ({depenses:,.0f} FCFA) dépassent "
            f"le seuil d'alerte ({BUDGETALERTE:,.0f} FCFA) !"
        )

def rafraichir_tableau():
    for ligne in tableau.get_children():
        tableau.delete(ligne)
    for i, t in enumerate(transactions):
        signe = "+" if t["type"] == "revenu" else "-"

```

```

montant_str = f"{signe}{t['montant']:,.0f}"
item = tableau.insert("", tk.END, iid=str(i),
                      values=(t["description"], montant_str, t["date"]))
tableau.item(item, tags=(t["type"],))
tableau.tag_configure("revenu", foreground="#50fa7b")
tableau.tag_configure("depense", foreground="#ff5555")

def rafraichir_graphiques():
    ax1.clear()
    ax2.clear()
    ax1.set_facecolor("#2a2a3e")
    ax2.set_facecolor("#2a2a3e")

    dependses_par_cat = {}
    for t in transactions:
        if t["type"] == "depense":
            cat = t["categorie"]
            dependses_par_cat[cat] = dependses_par_cat.get(cat, 0) + t["montant"]

    if dependses_par_cat:
        couleurs = ["#ff5555", "#ff79c6", "#bd93f9", "#6272a4",
                    "#ffb86c", "#50fa7b", "#8be9fd", "#f1fa8c",
                    "#ff6e6e", "#caa9fa", "#69ff47"]
        wedges, texts, autotexts = ax1.pie(
            dependses_par_cat.values(),
            labels=dependses_par_cat.keys(),
            autopct="%1.0f%%",
            colors=couleurs[:len(dependses_par_cat)],
            textprops={"color": "#cdd6f4", "fontsize": 7},
            startangle=90
        )
        for at in autotexts:
            at.set_color("#f8f8f2")
            at.set_fontsize(7)
        ax1.set_title("Dépenses par catégorie",
                      color="#cdd6f4", fontsize=9, pad=8)
    else:
        ax1.text(0.5, 0.5, "Aucune dépense", ha="center", va="center",
                 color="#6272a4", fontsize=10, transform=ax1.transAxes)
        ax1.set_title("Dépenses par catégorie", color="#cdd6f4", fontsize=9)

    revenus, depenses, _ = calculer_solde()
    barres = ax2.bar(["Revenus", "Dépenses"], [revenus, depenses],
                     color=["#50fa7b", "#ff5555"], width=0.5, edgecolor="none")
    max_val = max(revenus, depenses, 1)
    for barre in barres:
        h = barre.get_height()
        ax2.text(barre.get_x() + barre.get_width() / 2,
                  h + max_val * 0.02,
                  f"{h:,.0f}", ha="center", va="bottom",
                  color="#cdd6f4", fontsize=8)
    ax2.set_title("Revenus vs Dépenses (FCFA)", color="#cdd6f4", fontsize=9)
    ax2.tick_params(colors="#cdd6f4", labelsize=8)

```

```

    for spine in ax2.spines.values():
        spine.set_edgecolor("#44475a")

figure.tight_layout(pad=1.5)
canvas.draw()

def ajouter_transaction():
    description = var_description.get().strip()
    montant_str = var_montant.get().strip()
    categorie = var_categorie.get()
    type_t = var_type.get()

    if not description:
        messagebox.showerror("Erreur", "Veuillez saisir une description.")
        return
    try:
        montant = float(montant_str)
        if montant <= 0:
            raise ValueError
    except ValueError:
        messagebox.showerror("Erreur", "Le montant doit être un nombre positif.")
        return

    transactions.append({
        "description": description,
        "montant": montant,
        "type": type_t,
        "categorie": categorie,
        "date": str(date.today())
    })

    rafraichir_tableau()
    mettre_a_jour_solde()
    rafraichir_graphiques()
    sauvegarder_donnees()
    var_description.set("")
    var_montant.set("")
    champ_description.focus()

def supprimer_transaction():
    selection = tableau.selection()
    if not selection:
        messagebox.showinfo("Info", "Sélectionnez une transaction à supprimer.")
        return
    if not messagebox.askyesno("Confirmer", "Supprimer cette transaction ?"):
        return
    transactions.pop(int(selection[0]))
    rafraichir_tableau()
    mettre_a_jour_solde()
    rafraichir_graphiques()
    sauvegarder_donnees()

```

```
def sauvegarder_donnees():
    try:
        with open(FICHIER_DONNEES, "w", encoding="utf-8") as f:
            json.dump(transactions, f, ensure_ascii=False, indent=2)
    except Exception as e:
        messagebox.showerror("Erreur", f"Sauvegarde impossible :\n{e}")

def charger_donnees():
    if not os.path.exists(FICHIER_DONNEES):
        return
    try:
        with open(FICHIER_DONNEES, "r", encoding="utf-8") as f:
            for t in json.load(f):
                transactions.append(t)
    except Exception as e:
        messagebox.showerror("Erreur", f"Chargement impossible :\n{e}")

# =====
#  INITIALISATION
#  =====

charger_donnees()
rafraichir_tableau()
mettre_a_jour_solde()
rafraichir_graphiques()

fenetre.mainloop()
```

✓ Vérification finale — Application complète

Votre application doit :

Fonctionnalité	Résultat attendu
Ajout d'un revenu	Ligne verte dans le tableau, solde augmente, barre verte monte
Ajout d'une dépense	Ligne rouge, solde diminue, camembert se met à jour
Dépenses > 1000 FCFA	Alerte popup automatique
Suppression	Sélectionner une ligne + cliquer Supprimer → confirmation → disparaît
Fermer et rouvrir	Toutes les transactions réapparaissent
Solde négatif	Label solde devient rouge

Problèmes courants et solutions

Erreur	Cause probable	Solution
--------	----------------	----------

Erreur	Cause probable	Solution
<code>ModuleNotFoundError: matplotlib</code>	Non installé	<code>pip install matplotlib</code>
<code>NameError: name 'ajouter_transaction' is not defined</code>	Fonction définie après le bouton	Utiliser <code>command=lambda: ajouter_transaction()</code>
Le graphique ne s'affiche pas	<code>canvas.draw()</code> manquant	Vérifiez l'appel en fin de <code>rafraichir_graphiques()</code>
Les accents ne s'affichent pas dans le JSON	Encodage manquant	Ajouter <code>encoding="utf-8"</code> dans <code>open()</code>
La fenêtre se referme immédiatement	<code>mainloop()</code> manquant	Vérifiez la dernière ligne du fichier

Axes d'amélioration — Vers le niveau intermédiaire et avancé

Voici des fonctionnalités à ajouter pour approfondir vos compétences Python :

Niveau intermédiaire

1. Filtre par période (dates)

Concept : manipulation de dates avec le module `datetime`

Défi : ajouter deux champs "du" et "au" et filtrer les transactions affichées

2. Modifier une transaction existante

Concept : mise à jour d'un élément dans une liste, préremplissage d'un formulaire

Défi : double-clic sur une ligne ouvre le formulaire avec les données préremplies

3. Exporter en CSV

```
# Exemple de départ
import csv
with open("export.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=
["description", "montant", "type", "categorie", "date"])
    writer.writeheader()
    writer.writerows(transactions)
```


4. Budget par catégorie

Concept : dictionnaire de budgets, comparaison et alerte par catégorie
Défi : permettre de définir un budget maximum pour "Alimentation", "Loisirs"...

5. Graphique d'évolution mensuelle

Concept : regroupement de données par mois avec `collections.defaultdict`
Défi : afficher un graphique en courbe de l'évolution du solde dans le temps

Niveau avancé

6. Base de données SQLite au lieu du JSON

```
# Remplacer json par sqlite3 (inclus dans Python)
import sqlite3
conn = sqlite3.connect("budget.db")
# Concept : SQL, CREATE TABLE, INSERT, SELECT, DELETE
```

7. Interface multi-fenêtres (Toplevel)

Concept : fenêtre secondaire avec `tk.Toplevel`
Défi : créer une fenêtre "Paramètres" pour modifier le budget d'alerte et les catégories

8. Thèmes clair/sombre dynamique

Concept : dictionnaire de couleurs, fonction qui rebascule tous les widgets
Défi : bouton qui alterne entre un thème sombre (`#1e1e2e`) et clair (`#f8f8f2`)

9. Application web avec Flask

Concept : transformer l'app desktop en application web accessible depuis un navigateur
Outils : Flask (`pip install flask`), HTML/CSS de base, JSON comme base de données

10. Packaging — distribuer l'application

```
# Créer un exécutable .exe (Windows) ou binaire (Linux/Mac)
pip install pyinstaller
pyinstaller --onefile --windowed app.py
# Concept : packaging, dépendances, distribution sans Python installé
```

Récapitulatif des concepts maîtrisés

Concept Python	Où vous l'avez utilisé
Variables et types	Montants, descriptions, types de transactions
Listes et dictionnaires	La liste <code>transactions</code> , chaque transaction = dict
Fonctions	<code>ajouter_transaction()</code> , <code>rafraichir_graphiques()</code> ...
Conditions <code>if/else</code>	Validation du formulaire, couleur du solde
Boucles <code>for</code>	Parcours des transactions pour affichage et calcul
Gestion d'erreurs <code>try/except</code>	Validation montant, lecture/écriture fichier
Modules	<code>tkinter</code> , <code>matplotlib</code> , <code>json</code> , <code>os</code> , <code>datetime</code>
Callbacks/événements	<code>command=</code> sur les boutons
Lecture/écriture fichier	<code>open()</code> , <code>json.dump()</code> , <code>json.load()</code>
Compréhensions de liste	<code>sum(t["montant"] for t in transactions if ...)</code>

Félicitations !

Vous venez de construire une application de bureau complète et fonctionnelle.

Ce que vous avez réalisé ici — une interface graphique, des graphiques dynamiques, la persistance des données — est exactement ce que l'on trouve dans des logiciels professionnels, simplement à plus grande échelle.

La différence entre ce projet et une vraie application commerciale, c'est la quantité de fonctionnalités — pas les concepts fondamentaux, que vous maîtrisez désormais.